*** DSA Using JAVA ***

Quick Array Revision - few good problems only
Sorting Methods   - Merge and Quick sort
Strings / stringbuilder
Collections
(ArrayList, HashMap, HashSet)
Stack
Queue
Priority Queue
Recursion & Backtracking
Linked List
Trees
Graphs
Dynamic Programming
Greedy
Bit Manipulation

Arrays

1.Intro
Array search

2.Basic traversal
missing number
Check if Array is Sorted
    These build loop control, indexing, and simple linear scan confidence.

3.Two pointers
Reverse an array (in-place).
Remove duplicates from sorted array.
Move zeros to end (stable).
Merge two sorted arrays (in-place).
Count pairs with given sum / 3Sum
Trapping Rain Water.
Container With Most Water
Squares of a Sorted Array
Remove Element
Dutch National Flag (Sort Colors)
Rotate array by k positions. ☑
   These are the core sorted-array and boundary-moving patterns that show up again and again in interviews.

4.Prefix sum / suffix
Range Sum Query
Difference Array

Product Except Self
Corporate Flight Bookings
These problems build the habit of using precomputed left/right information instead of recomputing everything

5.Sliding window
Subarray with given sum (positive numbers).
Longest subarray with at most K distinct (or exactly K distinct).
Product subarrays less than K.
Maximum Sum Subarray of Size K
First Negative Integer in Every Window
Sliding window maximum (deque).
    These teach fixed-size and variable-size window logic for subarray-based problems.

6.Kadane / subarray DP
Kadane’s maximum subarray sum.
Maximum product subarray.
Maximum sum circular subarray.
Maximum Absolute Sum
These teach local-best and global-best thinking, especially when negatives or wrap-around are involved.

7.Binary search
Lower Bound
Upper Bound
Search Insert Position
First & Last Occurrence
Peak Element
Search in Rotated Array
Find Minimum in Rotated Array
Single Element in Sorted Array
Median of Two Sorted Arrays
These are the main “array + order property” problems where binary search is adapted beyond plain lookup.

8.Sorting + sweep
Merge intervals / interval intersection (array-of-pairs).
Minimum swaps to sort an array.
Next permutation.
Count inversions (merge-sort).
Meeting Rooms
Insert Interval
Non-overlapping Intervals
 These are important because sorting often unlocks the cleanest solution path.

9.Index-as-hash / constructive

Find duplicate / missing number (one of each).
Set Mismatch
Find All Missing Numbers
Find All Duplicates
Smallest missing positive (first missing positive).
Reconstruct array from differences / permutation from given constraints (pattern problems).
These are excellent for learning in-place marking, permutation cycles, and constructive reasoning.

10. Matrix (2D Arrays)
Matrix
Spiral Matrix
Rotate Image
Set Matrix Zeroes
Search a 2D Matrix
Word Search
Number of Islands (BFS/DFS application)

SORTING - merge , quick

STRING

STRINGBUILDER

COLLECTIONS

ARRAYLIST
Remove duplicates from sorted array.

Hash map / set
Frequency of Elements ⋆
Count Distinct Elements ⋆
Find First Repeating Element
Find First Non-Repeating Element
Two Sum ⋆⋆⋆
Count Pairs with Given Sum ⋆⋆⋆
Contains Duplicate
Majority Element
Valid Anagram
Intersection of Two Arrays

Longest Subarray Sum = K
Top K Frequent Elements
Longest Subarray with Sum 0
Prefix-sum & Subarray Sum Equals K.
Count subarrays with sum = K (using prefix-hash) and Count subarrays with at most K distinct.
Longest consecutive sequence.
   These are the main lookup/frequency patterns used to convert brute force into linear-time solutions.

.....